LABORATORY PROGRAM – 10

Write a simple streaming program in Spark to receive text data streams on a particular port, perform basic text cleaning (like white space removal, stop words removal, lemmatization, etc.), and print the cleaned text on the screen. (Open Ended Question).

CODE, COMMAND WITH OUTPUT

```
# Install NLTK and download required data (run once)
!pip install nltk

import nltk
nltk.download('punkt'
)
nltk.download('stopw
ords')
nltk.download('wordn
et')

from pyspark.sql import SparkSession
from pyspark.sql.functions import col, lower,
regexp_replace, split, explode, udf from pyspark.sql.types
import ArrayType, StringType
from pyspark.ml.feature import
StopWordsRemover from nltk.stem
import WordNetLemmatizer

# Initialize SparkSession
spark = SparkSession.builder.appName("TextProcessing").getOrCreate()

# Define your
input lines lines =
[
  "Hello, I hate you.",
  "I hate that I love you.",
  "Don't want to, but I
  can't put", "nobody
  else above you."
]
```

```
# Create DataFrame from lines
df = spark.createDataFrame(lines, "string").toDF("value")

# Step 1: Lowercase and remove punctuation
df_clean = df.select(regexp_replace(lower(col("value")), "[^a-zA-Z\\s]", "").alias("cleaned"))

# Step 2: Tokenize the cleaned text
df_tokens = df_clean.select(split(col("cleaned"), "\\s+").alias("tokens"))

# Step 3: Remove stop words
remover = StopWordsRemover(inputCol="tokens",
outputCol="filtered") df_filtered =
remover.transform(df_tokens)

# Step 4: Lemmatization using NLTK
WordNetLemmatizer with UDF lemmatizer =
WordNetLemmatizer()

def lemmatize_words(words):
   return [lemmatizer.lemmatize(word) for word

in words] lemmatize_udf =

udf(lemmatize_words,

ArrayType(StringType()))

df_lemmatized = df_filtered.withColumn("lemmatized", lemmatize_udf(col("filtered")))


# Step 5: Explode the lemmatized words and show results
df_lemmatized.select(explode(col("lemmatized")).alias("word")).show(truncate=False)
```

```
Session  Actions  Edit  View  Help
  Downloading joblib-1.5.3-py3-none-any.whl.metadata (5.5 kB)
Collecting regex≥2021.8.3 (from nltk)
  Downloading regex-2026.4.4-cp313-cp313-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata (40 kB)
Collecting tqdm (from nltk)
  Downloading tqdm-4.67.3-py3-none-any.whl.metadata (57 kB)
Downloading nltk-3.9.4-py3-none-any.whl (1.6 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.6/1.6 MB 4.4 MB/s  0:00:00
Downloading regex-2026.4.4-cp313-cp313-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (801 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 801.1/801.1 kB 4.2 MB/s  0:00:00
Downloading click-8.3.3-py3-none-any.whl (110 kB)
Downloading joblib-1.5.3-py3-none-any.whl (309 kB)
Downloading tqdm-4.67.3-py3-none-any.whl (78 kB)
Installing collected packages: tqdm, regex, joblib, click, nltk
Successfully installed click-8.3.3 joblib-1.5.3 nltk-3.9.4 regex-2026.4.4 tqdm-4.67.3

┌──(spark_env)─(thilak㉿kali)-[~]
└─$ python
Python 3.13.12 (main, Feb  4 2026, 15:06:39) [GCC 15.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import nltk
... nltk.download('punkt')
... nltk.download('stopwords')
... nltk.download('wordnet')
... exit()
...
[nltk_data] Downloading package punkt to /home/thilak/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package stopwords to /home/thilak/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
[nltk_data] Downloading package wordnet to /home/thilak/nltk_data...

┌──(spark_env)─(thilak㉿kali)-[~]
└─$ PYSPARK_PYTHON=~/spark_env/bin/python pyspark
Python 3.13.12 (main, Feb  4 2026, 15:06:39) [GCC 15.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
26/05/02 15:16:19 WARN Utils: Your hostname, kali resolves to a loopback address: 127.0.1.1; using 10.0.2.15 instead (on interface eth0)
26/05/02 15:16:19 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/05/02 15:16:21 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
^[[B^[[B^[[B^[[B^[[B^[[B^[[B^[[B^[[B^[[B^[[B^[[B^[[B^[[B^[[B^[[B^[[B^[[B^[[B^[[B^[[B^[[B^[[B^[[B^[[B^[[B^[[B^[[B^[[BWelcome to

      ____              __
     / __/__  ___ _____/ /__
    _\ \/ _ \/ _ `/ __/  '_/
   /__ / .__/\_,_/_/ /_/\_\   version 3.5.0
      /_/

Using Python version 3.13.12 (main, Feb  4 2026 15:06:39)
Spark context Web UI available at http://10.0.2.15:4040
Spark context available as 'sc' (master = local[*], app id = local-1777715182784).
SparkSession available as 'spark'.
>>>
```

```
Session  Actions  Edit  View  Help
>>> from pyspark.sql.functions import col, lower, regexp_replace, split, explode, udf
... from pyspark.sql.types import ArrayType, StringType
... from nltk.stem import WordNetLemmatizer
...
... lines = [
...     "Hello, I hate you.",
...     "I hate that I love you.",
...     "Don't want to, but I can't put",
...     "nobody else above you."
... ]
...
... df = spark.createDataFrame(lines, "string").toDF("value")
...
... df_clean = df.select(
...     regexp_replace(lower(col("value")), "[^a-zA-Z\\s]", "").alias("cleaned")
... )
...
... df_tokens = df_clean.select(
...     split(col("cleaned"), "\\s+").alias("tokens")
... )
...
... stopwords = {"i","you","that","to","but","the","a","an","is","are"}
...
... def remove_stopwords(words):
...     return [w for w in words if w not in stopwords]
...
... remove_udf = udf(remove_stopwords, ArrayType(StringType()))
... df_filtered = df_tokens.withColumn("filtered", remove_udf(col("tokens")))
...
... lemmatizer = WordNetLemmatizer()
...
... def lemmatize(words):
...     return [lemmatizer.lemmatize(word) for word in words]
...
... lemmatize_udf = udf(lemmatize, ArrayType(StringType()))
... df_lemmatized = df_filtered.withColumn("lemmatized", lemmatize_udf(col("filtered")))
...
>>> for row in df_lemmatized.select(explode(col("lemmatized")).alias("word")).collect():
...     print(row)
...
Row(word='hello')
Row(word='hate')
Row(word='hate')
Row(word='love')
Row(word='dont')
Row(word='want')
Row(word='cant')
Row(word='put')
Row(word='nobody')
Row(word='else')
Row(word='above')
>>>
```